

PolyformTown

Extended Technical Summary — V5.0

A lattice polyform enumeration engine for hat-tiling research

May 21, 2026

Project Overview

PolyformTown is a research-grade C engine that enumerates *lattice polyforms* — connected shapes built by joining tiles along shared edges — and validates counts against OEIS reference sequences. The ultimate research goal is to prove properties of the **hat tiling**, an aperiodic monotile discovered in 2023.

The engine supports three lattice types with a single shared geometric core, and refines results through a layered *run-level* (RL) pipeline that combines combinatorial enumeration, canonical hashing, and SAT-based verification.

Lattice	Symmetry group	Example tiles
Square	D_4 (8 transforms)	monomino, domino, chair
Triangular	D_6 (12 transforms)	triangle, hexagon, hh
Tetrille	D_6 tagged	kite, hat

Some OEIS sequences the engine validates against:

Sequence	Description	Lattice
A000104	Polyominoes without holes	Square
A000105	Free polyominoes	Square
A000577	Polyiamonds	Triangular
A000228	Polyhexes	Triangular

Repository Layout

Directory Structure

The repository root contains a nested PolyformTown/ project directory. All source, data, tiles, and test artefacts live inside that subdirectory.

src/core/

Geometry primitives: cycles, tiles, attachment, hashing, lattice transforms.

src/throughput/

Main growth pipelines: poly_pipeline, vcomp.

src/r10/~src/r16/

Run-level modules; each refines the prior level's output.

src/usage/

CLI tools: poly_count, poly_print, vcomp_count, depict tools.

tiles/

Tile DSL files (.tile).

data/

Generated artefacts (not committed).

tests/

Smoke and regression scripts.

manual/

Documentation and theory notes.

```
PolyformTown/
+-- src/
|   +-- core/          # Coord, Cycle, Poly, Edge,
|   |                  #   Tile, Hash, Lattice
|   +-- throughput/    # poly_pipeline, vcomp
|   +-- r10/ .. r16/   # run-level modules
|   +-- meta/          # dependency scanner
|   +-- usage/         # poly_count, poly_print ...
+-- tiles/
|   +-- hat.tile
|   +-- monomino.tile
|   +-- hexagon.tile ...
+-- data/
|   +-- r10/ r11/ r15/ ...
+-- tests/
|   +-- smoke.sh
|   +-- comprehensive.sh
+-- Makefile
+-- README.md
```

Core Data Structures

Coordinate and Cycle

Coordinate, Cycle, Poly, Edge — src/core/cycle.h

Every vertex in every lattice is represented by a **Coord**. The integer field *v* carries a *valence tag* ($v \in \{6, 4, 3\}$) used exclusively in the tetrille lattice; for square and triangular lattices it is set to 4 or 6 respectively and acts as a system identifier.

A **Cycle** is a closed polygon boundary stored as an ordered vertex array. A **Poly** packages one outer cycle plus zero or more hole cycles.

- Outer boundary: *counterclockwise* (CCW)
- Hole boundaries: *clockwise* (CW)
- Maximum 384 vertices per cycle, 32 cycles per poly

The **signed area** (shoelace formula) determines orientation: positive $\Rightarrow$ CCW (outer), negative $\Rightarrow$ CW (hole).

```
/* cycle.h */
typedef struct {
    int v; /* valence tag: 6/4/3 */
    int x, y;
} Coord;

typedef struct {
    int n; /* vertex count */
    Coord v[MAX_VERTS]; /* MAX_VERTS = 384 */
} Cycle;

typedef struct {
    int cycle_count;
    Cycle cycles[MAX_CYCLES]; /* [0]=outer, rest=holes
    ↪ */
} Poly;

typedef struct {
    Coord a, b; /* directed edge a→b */
} Edge;

/* Shoelace area (x2) - sign gives orientation */
long long cycle_signed_area2(
    const Cycle *c, int lattice);
```

Tile

Tile — src/core/tile.h

A **Tile** bundles the base boundary with all its pre-computed symmetry variants and the geometric metadata needed for SVG output.

base Canonical boundary loaded from the `.tile` file.

variants Up to 12 symmetry images (rotations and reflections). Computed once by `tile_build_variants()`.

constants Named symbolic constants, e.g. $r = \sqrt{3}$, used in the basis matrix expressions.

bases Per-valence 2×2 basis matrices that map integer lattice coordinates to real-plane coordinates for rendering.

Tile files use a simple DSL listing **cycle** vertices, optional **constants**, optional **basis** matrices, and the **lattice** type (`square`, `triangular`).

```
/* tile.h */
#define TILE_LATTICE_SQUARE      0
#define TILE_LATTICE_TRIANGULAR  1
#define TILE_LATTICE_TETRILLE    2
#define MAX_VARIANTS 12

typedef struct {
    char name[64];
    Cycle base;
    int lattice;
    int variant_count;
    Cycle variants[MAX_VARIANTS];
    int constant_count;
    TileConstant constants[16];
    int basis_count;
    TileBasis bases[16];
} Tile;

/* hat.tile (excerpt) */
// lattice: tetrille
// constants: r = sqrt(3)
// basis 6:  r/2  -1/2
//           r/2   1/2
// cycle:
//   6 0 0   4 -1 0   3 -1 0 ...
```

Hash Table

Open-addressing Hash Table — src/core/hash.h / hash.c

After canonicalization each polyform is inserted into a **HashTable** to ensure each symmetry class is counted exactly once across the growth level.

The hash function is FNV-1a over all cycle vertices (valence tag, x , y). Collision resolution uses linear probing; the table doubles when load exceeds 70%.

`hash_insert()` returns 1 if the shape was new (and stores it), or 0 if it was already present.

```
/* hash.h */
typedef struct { Poly p; int used; } HashSlot;
typedef struct {
    HashSlot *slots;
    size_t size, count;
} HashTable;

/* hash.c - FNV-1a over vertex triples */
static unsigned poly_hash(const Poly *p) {
    unsigned h = 2166136261u;
    h ^= (unsigned)p->cycle_count;
    h *= 16777619u;
    for (int k = 0; k < p->cycle_count; k++) {
        const Cycle *c = &p->cycles[k];
        for (int i = 0; i < c->n; i++) {
            h ^= (unsigned)(c->v[i].v + 10000);
            h *= 16777619u;
            h ^= (unsigned)(c->v[i].x + 10000);
            h *= 16777619u;
            h ^= (unsigned)(c->v[i].y + 10000);
            h *= 16777619u;
        }
    }
    return h;
}

/* resize doubles table, re-hashes all entries */
/* insert returns 1=new, 0=duplicate */
int hash_insert(HashTable *h, const Poly *p);
```

Lattice Geometry

Three Lattice Types

Lattice Abstraction — `src/core/lattice.h`, `tetrille.h`

The three lattice types share a common interface so that the attachment and canonicalization algorithms are lattice-agnostic. Key per-lattice quantities:

Transform count Number of distinct symmetry group elements.

Direction count Coordination number of each vertex.

Embed point Maps integer lattice coordinates to the real plane (used only for SVG rendering and tie-breaking, never for enumeration).

The **tetrille** lattice uses *tagged coordinates*. Each vertex carries a type tag $v \in \{6, 4, 3\}$ corresponding to the three vertex valences of the kite-and-dart decomposition. Translations must be lifted through the 6-system using exact integer formulas to avoid cross-tag arithmetic errors.

```
/* lattice.h */
int lattice_transform_count(int lattice);
int lattice_direction_count(int lattice);
void lattice_embed_point(int lattice,
    Coord p, double *x, double *y);

/* tetrille.h - tagged translation in 6-system */
void tetrille_translate_cycle(
    Cycle *c, int m6, int n6);

/* cycle.c - tag-aware translation */
static void cycle_translate_tetrille(
    Cycle *c, int m6, int n6) {
    for (int i = 0; i < c->n; i++) {
        Coord *p = &c->v[i];
        if (p->v == 6) { p->x += m6; p->y += n6; }
        else if (p->v == 4) { p->x += 2*m6; p->y += 2*n6; }
        else if (p->v == 3) {
            p->x += 2*m6 + n6;
            p->y += -m6 + n6;
        }
    }
}
```

Signed Area and Orientation

Shoelace Formula — `src/core/cycle.c`

The signed area of a cycle is used to determine its orientation and to identify the outer boundary (maximum positive area) versus holes (negative area) after each tile attachment.

For non-tetrille lattices the shoelace formula is applied directly to integer coordinates. For the tetrille lattice each vertex is first embedded in a scaled real plane (scale factor 6) using fixed integer arithmetic to avoid floating-point errors.

$$A_2 = \sum_{i=0}^{n-1} (x_i y_{i+1} - x_{i+1} y_i)$$

$A_2 > 0$: CCW (outer boundary); $A_2 < 0$: CW (hole).

```
/* cycle.c */
long long cycle_signed_area2(
    const Cycle *c, int lattice) {
    int tet = (lattice == TILE_LATTICE_TETRILLE);
    long long s = 0;
    for (int i = 0; i < c->n; i++) {
        Coord a = c->v[i];
        Coord b = c->v[(i + 1) % c->n];
        if (tet) {
            long long ax, ay, bx, by;
            tetrille_embed6_scaled(a, &ax, &ay);
            tetrille_embed6_scaled(b, &bx, &by);
            s += ax * by - bx * ay;
        } else {
            s += (long long)a.x * b.y
                - (long long)b.x * a.y;
        }
    }
    return s;
}

/* s > 0 + CCW (outer) */
/* s < 0 + CW (hole) */
```

Polyform Enumeration Pipeline

Growth Loop

Core Growth Loop — src/throughput/poly_pipeline.c

The growth loop is the heart of the engine. Starting from the canonical seed (a single tile), it iterates level by level:

1. Collect all boundary edges of every polyform in the current level.
2. For each tile variant, try to attach it at each (frontier-edge, tile-edge) pair.
3. Canonicalize the result under the lattice symmetry group.
4. Filter with the live-boundary test (optional).
5. Insert into a hash table; only new canonical shapes advance to the next level.

A callback `on_level()` fires at the start of each level and can terminate the loop early, making the pipeline reusable for both counting and printing tools.

```
/* poly_pipeline.c */
void run_poly_levels(const Tile *tile,
    int max_n, int live_only,
    PolyLevelFn on_level, void *ud) {

    Poly seed_raw, seed;
    seed_raw.cycle_count = 1;
    seed_raw.cycles[0] = tile->base;
    poly_hash_key_lattice(&seed_raw,
        tile->lattice, &seed);

    PolyVec cur, next;
    vec_init(&cur, 64); vec_init(&next, 64);
    vec_push(&cur, &seed);

    for (int lv = 1; lv <= max_n; lv++) {
        if (on_level && !on_level(lv, &cur, tile, ud))
            break;
        if (lv == max_n) break;

        HashTable seen;
        hash_init(&seen, 1024);
        vec_clear(&next);

        for (size_t i = 0; i < cur.count; i++) {
            Edge fr[MAX_VERTS * MAX_CYCLES];
            int fc = build_boundary_edges(
                &cur.data[i], fr);
            for (int v = 0; v < tile->variant_count; v++) {
                const Cycle *tv = &tile->variants[v];
                for (int be = 0; be < fc; be++)
                    for (int te = 0; te < tv->n; te++) {
                        Poly grown, canon;
                        if (!try_attach_tile_poly(&cur.data[i],
                            tv, tile->lattice, be, te, &grown))
                            continue;
                        poly_hash_key_lattice(&grown,
                            tile->lattice, &canon);
                        if (live_only &&
                            !poly_has_live_boundary(&canon, tile))
                            continue;
                        if (hash_insert(&seen, &canon))
                            vec_push(&next, &canon);
                    }
            }
        }
        hash_destroy(&seen);
        PolyVec tmp = cur; cur = next; next = tmp;
    }
}
```

Tile Attachment

Attachment Algorithm — `src/core/attach.c`

`try_attach_tile_poly_ex_status()` is the geometric workhorse. Given a frontier edge on the existing polyform and an edge of a tile variant, it:

1. **Aligns** the tile so the chosen tile edge is antiparallel and coincident with the chosen frontier edge. For tetrille this requires lifting the translation into the 6-system.
2. **Merges** all directed edges from both the polyform and the aligned tile into a flat list.
3. **Cancels** antiparallel coincident edge pairs (the shared edge disappears from the boundary). Parallel coincident edges indicate an illegal overlap and abort.
4. **Walks** the remaining edges to extract boundary cycles via a left-first (outer) or right-first (hole) traversal.
5. **Classifies** cycles by signed area and checks that no interior intersection occurs.

Return codes (`AttachStatus`) distinguish geometry errors, buffer overflows, and valid results.

```
/* attach.h */
typedef enum {
    ATTACH_STATUS_OK           = 1,
    ATTACH_STATUS_GEOMETRY     = 0, /* overlap */
    ATTACH_STATUS_BOUNDARY_BOUND = -1, /* too many edges */
    ↪ /*
    ATTACH_STATUS_CYCLE_BOUND  = -2, /* walk failed */
    ATTACH_STATUS_INTERNAL_BOUND = -3 /* interior hit */
    ↪ /*
} AttachStatus;

/* attach.c - edge cancellation core */
static int build_union_edges(
    const Poly *a, const Cycle *b,
    LEdge *out, int *out_n,
    AttachStatus *status) {
    /* collect a's edges, then b's edges */
    for (int i = 0; i < n; i++) {
        if (out[i].canceled) continue;
        for (int j = i+1; j < n; j++) {
            if (out[j].canceled) continue;
            if (edge_same(out[i].e, out[j].e)) {
                /* parallel duplicate → invalid */
                attach_status_set(status,
                    ATTACH_STATUS_GEOMETRY);
                return 0;
            }
            if (edge_opp(out[i].e, out[j].e)) {
                /* antiparallel → cancel both */
                out[i].canceled = 1;
                out[j].canceled = 1;
                break;
            }
        }
    }
    return 1;
}
```

Cycle Walking

Directed Cycle Extraction — `src/core/attach.c`

After edge cancellation the remaining directed edges form one or more closed cycles. The walker picks the globally-lowest starting edge (by embedded position), then follows a *prefer-left* path to trace the outer boundary and a *prefer-right* path for holes. Direction scoring is modular-arithmetic over the lattice direction count, so it handles all three lattice types uniformly.

```
/* attach.c - direction-scored next-edge picker */
static int pick_next_edge(
    const Edge *edges, int m,
    const int *used, Coord v,
    int prev_dir, int prefer_left,
    int lattice) {
    int dc = lattice_direction_count(lattice);
    int rev = (prev_dir + dc/2) % dc;
    int best = -1;
    int best_score = prefer_left ? -1 : dc+1;

    for (int i = 0; i < m; i++) {
        if (used[i]) continue;
        if (!coord_eq(edges[i].a, v)) continue;
        int d = lattice_direction_index(
            lattice, edges[i]);
        if (d < 0) continue;
        int score = (d - rev + dc) % dc;
        if (score == 0) continue;
        if (prefer_left ? (score > best_score)
            : (score < best_score)) {
            best_score = score; best = i;
        }
    }
    return best; /* -1 if no valid edge found */
}
```

Canonical Form — src/core/cycle.c/cycle.h

Two polyforms are *equivalent* if one can be obtained from the other by a lattice symmetry (rotation and/or reflection). To count equivalence classes rather than labelled shapes, every new polyform is reduced to its **canonical representative**:

1. Apply all symmetry transforms ($|D_4|=8$ or $|D_6|=12$ elements).
2. For each image, normalize position (translate so the lexicographically smallest vertex is at the origin).
3. Rotate each cycle to its lexicographically minimal starting vertex.
4. Sort hole cycles lexicographically.
5. Select the image that is lexicographically smallest overall.

`poly_hash_key_lattice()` wraps this into the one call made by the growth loop before every hash insert.

```
/* cycle.h */
void poly_hash_key_lattice(
    const Poly *src, int lattice, Poly *key);

/* Internally calls poly_canonicalize_lattice(),
   which applies all lattice transforms and
   keeps the lexicographic minimum. */

void poly_canonicalize_lattice(
    const Poly *src, Poly *out, int lattice) {
    int nt = lattice_transform_count(lattice);
    Poly best;
    int have_best = 0;
    for (int t = 0; t < nt; t++) {
        Poly tmp;
        poly_transform_lattice(src, &tmp,
                               lattice, t);
        poly_normalize_position(&tmp, lattice);
        /* shift each cycle to its min vertex */
        for (int k = 0; k < tmp.cycle_count; k++)
            cycle_canonicalize_shift(
                &tmp.cycles[k]);
        /* sort hole cycles */
        /* compare to running best */
        if (!have_best || poly_less(&tmp, &best)) {
            best = tmp; have_best = 1;
        }
    }
    *out = best;
}
```

Vertex Completion (VComp)

Vertex-Completion Enumeration — src/throughput/vcomp.h

An alternative enumeration mode tracks how many distinct ways a specific **target vertex** can be completely surrounded by tiles. This corresponds to enumerating valid *vertex figures* and is critical for the run-level analysis.

The state records:

target The vertex being completed.

hidden Vertices discovered in the interior of the growing figure (constrained by the target's local geometry).

ports Exposed attachment sites on the boundary that can still receive tiles.

tiles History of attached tiles (when tracking is enabled).

Multiple levels record partial completions graded by the number of hidden vertices — a slowly-growing quantity that makes depth-limited search tractable.

```
/* vcomp.h */
typedef struct {
    Poly poly;          /* current boundary shape */
    Coord target;       /* vertex being completed */
    Coord hidden[VCOMP_MAX_HIDDEN];
    int hidden_count;
    Coord ports[VCOMP_MAX_HIDDEN]; /* open sites */
    int port_count;
    Cycle tiles[VCOMP_MAX_TRACE]; /* history */
    int tile_count;
} VCompRawState;

/* Enumerate all partial/complete vertex
   figures up to max_hidden interior verts */
void vcomp_enumerate_levels(
    const Poly *base,
    const Tile *tile,
    Coord target,
    const Coord *initial_hidden,
    int initial_hidden_count,
    const Coord *initial_ports,
    int initial_port_count,
    const Cycle *initial_tiles,
    int initial_tile_count,
    int max_hidden,
    int track_tiles,
    VCompLevels *out);
```

Run-Level Pipeline

The run-level system progressively narrows the space of hat-tiling configurations by applying successively stronger local constraints. Each level produces a *completions* dataset and optionally a *deletions* dataset consumed by the next level.

RL0: Vertex Figures

RL0 — Vertex Figure Generation — `src/r10/`

RL0 exhaustively enumerates all valid ways to tile completely around a single vertex. Results are stored in `data/r10/completions.dat` as multi-line records delimited by `--[N]--`.

The **ForgetMap** (remembrance dictionary) indexes every vertex figure by its *arc* — a cyclic sequence of (parity, tile-index) pairs — enabling rapid lookup of valid completions given any partial arc. This dictionary is the primary filter used by RL1.

The **deletion set** records vertex figures that are rejected by refinement passes (e.g. because they cannot be extended to a valid surround at the next level).

```
/* forget_map.h */
typedef struct { int p; int i; } RLOFItem;
/* p = parity ( $\pm 1$ ), i = tile variant index */

typedef struct {
    int n;
    RLOFItem item[16]; /* arc of (p,i) pairs */
} RLOFMCycle; /* also used as RLOFMarc */

typedef struct {
    RLOFMarc key;
    RLOFMarc *values;
    int value_count;
    int value_capacity;
} RLOFMRow;

typedef struct {
    RLOFMCycle *cycles;
    int cycle_count, cycle_capacity;
    RLOFMRow *rows;
    int row_count, row_capacity;
} RLOForgetMap;

/* Look up all valid completions for a partial arc */
int r10_fm_lookup(const RLOForgetMap *map,
                  const RLOFMarc *key,
                  const RLOFMarc **values,
                  int *value_count);
```

RL1–RL3: Surround Levels

RL1–RL3 — Boundary Completion Filters — src/rl1/ ...src/rl3/

Each surround level takes the completion records from the previous level and checks whether each vertex figure can be extended to a valid single (RL1), double (RL2), or triple (RL3) ring of surrounding tiles.

Extension uses a DFS-based **boundary completion** (**bcomp**) search that respects the RL0 remembrance dictionary: every partial vertex figure encountered during the extension must appear in the dictionary, or the branch is pruned immediately.

Records that *cannot* be extended are added to the deletions file, which is consumed by the next level's loader (via `-depth` refinement).

```
# Build pipeline (Makefile)
make boot          # RL0 + RL5 in sequence
make boot RL3_REFINE_DEPTH=2 # deeper pruning

# RL0
./bin/rl0_generate [tilefile]
./bin/rl0_refine

# RL1
./bin/rl1_generate \
  --rl0 data/rl0/completions.dat \
  --output data/rl1/completions.dat

./bin/rl1_refine \
  --input data/rl1/completions.dat \
  --output data/rl1/deletions.dat \
  --depth 1

# RL3 with deeper elimination
./bin/rl1_refine \
  --input data/rl3/completions.dat \
  --output data/rl3/deletions.dat \
  --depth 2
```

RL4: Supertile Extraction

RL4 — Supertile Candidates — src/rl4/

RL4 extracts candidate *supertile* shapes from the RL3-filtered surround set. A supertile is a second-order metatile whose internal structure is fully determined by the local hat-tiling constraints established in RL0–RL3.

Two supertile preferences are written:

`preferences/focus.supertile` Smaller default candidate used in subsequent RL5 analysis.

`preferences/overlap.supertile` Maximal overlap candidate for cross-checking.

```
# Regenerate RL4 data
make rl4_refine_data
make rl4_refine_svg

# Override supertile for RL5 extraction
make rl4_supertile_hexagons_data \
  RL4_SUPERTILE=preferences/overlap.supertile \
  RL4_SUPERTILE_HEXAGONS_DATA=\
    data/rl4/overlap_supertile_hexagons.dat
```

RL5: Hexagon Model and SAT Verification

RL5 — Hexagon Model — src/rl5/hex5.h

RL5 operates on an abstract **hexagon model**: each hat supertile is treated as a labelled hexagon, with edge labels encoding which orientations of the hat tile can be placed on each face. Adjacency constraints become edge-matching rules.

The `Hex5Model` holds:

- `tiles` — all canonical hexagon label sets.
- `oriented` — all symmetry images.
- `rule_a` / `rule_b` — edge-compatibility rules (which label on face *a* can be adjacent to label on face *b*).

`BComp5Result` stores complete boundary-completion rings found by the DFS over the hexagon constraint graph. The `Inner5Set` records pairs of hexagon labels that appear back-to-back in any completed ring — used as inner boundary constraints.

```
/* hex5.h */
#define HEX5_SIDES 6
typedef struct {
    char e[HEX5_SIDES][64]; /* 6 edge labels */
} Hex5;

typedef struct {
    Hex5 tiles[512];    int ntiles;
    Hex5 oriented[4096]; int noriented;
    char rule_a[4096][64];
    char rule_b[4096][64];
    int nrules;
} Hex5Model;

typedef struct {
    int center;
    int r[HEX5_SIDES]; /* ring of 6 tiles */
} Ring5;

typedef struct {
    Ring5 rings[262144]; int nrings;
    long starts, closure_tests, closure_hits;
} BComp5Result;

/* Build vertex-completion lookup */
int vcomp5_build(
    const Attach5Dict *a, VComp5Dict *v);

/* Build boundary-completion rings */
int bcomp5_build(const Hex5Model *m,
    const VComp5Dict *v, BComp5Result *b);
```

RL5 — SAT Verification — src/rl5/sat5.c

For pathological supertile configurations that cannot be eliminated by local constraint propagation alone, RL5 generates a SAT instance and calls an external solver (pysat / PySAT library) to determine satisfiability.

The SAT encoding maps each possible tile placement at each position to a Boolean variable; the local adjacency and vertex-completion constraints become clauses in CNF form. A SAT result of *unsatisfiable* proves that the configuration cannot appear in any valid hat tiling.

Output artefacts for both supertile candidates:

- `data/rl5/hexagons.dat`
- `data/rl5/supertile_hexagons.dat`
- `data/rl5/overlap_supertile_hexagons.dat`
- `data/rl5/equivalence_candidates.dat`

```
# Generate RL5 hexagon datasets
make rl5_generate

# Run equivalence scaffold
make rl5_refine_data

# Inspect candidates
less data/rl5/equivalence_candidates.dat

# Pipe SAT instance to solver (example)
./bin/rl5_refine \
    --hex data/rl5/hexagons.dat \
    --out data/rl5/equivalence_candidates.dat

# SAT invocation (internal)
# python3 -m pysat.examples.rc2 <cnf>
```

Tile File Format

Tile DSL — tiles/*.tile

Tile files are plain-text DSL files specifying the tile name, lattice type, optional symbolic constants ($\sqrt{3}$, etc.), optional rendering basis matrices, and the boundary cycle as a list of tagged integer coordinates.

The three example tiles below span all three lattice types. The hat tile uses all three coordinate valences in a single cycle, illustrating the complexity of the tetrille embedding.

```
# monomino.tile (square lattice)
name: monomino
lattice: square
basis:
4:
  1 0
  0 1
cycle:
4 0 0  4 1 0  4 1 1  4 0 1

# triangle.tile (triangular lattice)
name: triangle
lattice: triangular
constants:
r = sqrt(3)
basis:
6:
  1 0
  1/2 r/2
cycle:
6 0 0  6 1 0  6 0 1

# hat.tile (tetrille - three valence systems)
name: hat
lattice: tetrille
constants:
r = sqrt(3)
cycle:
6 0 0  4 -1 0  3 -1 0
4 -1 -1  6 0 -1  4 0 -3
3 -1 -2  4 1 -3  3 0 -2
4 1 -2  6 1 -1  4 2 -1
3 1 -1  4 1 -1
```

CLI Tools and Output

Usage Examples

The build produces several CLI tools in `bin/`. All tools accept an optional tile file argument and respect a `-live-only` flag that restricts output to polyforms whose boundary can still accept further tiles.

Output can be piped into `imgtable` to produce SVG grids, enabling visual inspection of enumerated shapes at any level.

Selected OEIS validations:

Count	n	Target
Monomino	7	107
Domino	5	4
Triangle	8	66
Hexagon	6	82

```
# Count polyforms (square lattice)
./bin/poly_count 7 tiles/monomino.tile
./bin/poly_count 5 tiles/domino.tile

# Count polyforms (triangular lattice)
./bin/poly_count 8 tiles/triangle.tile
./bin/poly_count 6 tiles/hexagon.tile

# Vertex-completion count (hat / tetrille)
./bin/vcomp_count 3 tiles/hat.tile

# Pipe to SVG renderer
./bin/vcomp_print 3 tiles/hat.tile \
  | ./bin/imgtable > out.svg

# RLO depiction grid
./bin/rlo_depict \
  --data data/rlo/completions.dat \
  --valence 6 --tile-count 3 --grouped \
  | ./bin/imgtable > img/rlo_v6_n3.svg

# RLO refinement graph
./bin/rlo_refine --print \
  | ./bin/depict_search0_graph \
  --colors > img/rlo_refine.svg
```

Testing

Test Suite — tests/

The project includes two test scripts that validate core functionality and count parity between the two enumeration modes (edge-attachment and vertex-completion).

Smoke test checks that:

- All CLI tools build and run without error.
- Basic polyomino counts match known OEIS values.
- Canonicalization is idempotent.

Comprehensive test additionally verifies:

- `vcomp_parity`: tracked and untracked `vcomp` enumeration yield identical counts for domino, chair, and hat.
- Hole-count checks via `poly_print` `grep`.
- RLO remembrance accounting consistency.

```
# Smoke test (fast)
bash tests/smoke.sh

# Full regression suite
bash tests/comprehensive.sh

# Expected hole counts from poly_print
# Monomino at level 7
./bin/poly_print 7 tiles/monomino.tile \
  | grep '^1 ' | wc -l # → 107

# Domino at level 4
./bin/poly_print 4 tiles/domino.tile \
  | grep '^1 ' | wc -l # → 3

# RLO remembrance count check
make check_rlo_remembrance_counts
# writes data/rlo/remembrance_counts.txt
```

Mathematical Background

Polyform Counting

A *polyform* of size n on a lattice is an equivalence class of connected unions of n tiles under the lattice symmetry group. The central algorithmic challenge is that the number of distinct polyforms grows exponentially in n , with a growth rate (connective constant) that depends on the tile geometry.

For polyominoes the growth constant is approximately $\lambda \approx 4.0626$, giving:

$$|\mathcal{P}_n| \sim C \cdot \lambda^n, \quad \lambda \approx 4.0626.$$

Edge Cancellation and Euler Characteristic

The boundary representation is closed under the edge-cancellation operation. After attaching a tile, the shared edge is cancelled; the remaining directed edges form a valid set of boundary cycles whose Euler characteristic satisfies:

$$V - E + F = 2 - 2g - h$$

where g is the genus and h the number of holes. In practice all polyforms have genus 0, so the formula reduces to $F = 2 - V + E + h$.

The Hat Tile and Aperiodicity

The **hat** is a 13-sided polygon on the tetrille (kite-and-dart) lattice whose discovery in 2023 provided the first known aperiodic monotile: a single shape that tiles the plane but only aperiodically.

PolyformTown’s run-level pipeline targets a computational proof of key local properties of the hat tiling, specifically that certain vertex figure constellations are forbidden, which would imply the aperiodicity constraint.

Architecture Summary

```
+-----+
|           Tile DSL  (.tile)           |
+-----+
|           | tile_load() / tile_build_variants()
+-----+
|           Core Layer  (src/core/)       |
| Coord Cycle Poly Edge Hash Lattice |
+-----+
|
|           +-----+
|           |           |
+-----+ +-----+
| poly_pipeline | | vcomp_pipeline |
| (edge growth  | | (vertex-figure  |
| + hash)       | | completion)     |
+-----+ +-----+
|           |           |
+-----+ +-----+
|           Run-Level Pipeline           |
| RL0 -> RL1 -> RL2 -> RL3 -> RL4 -> RL5 |
| (vertex (surrounds) (supertile)         |
| figures) (SAT verify)                   |
+-----+
```

PolyformTown V5.0 — stable, reproducible state with verified square, triangular, and tetrille workflows. SAT-based verification (RL5) and equivalence scaffold are current research frontiers. Compile with `pdflatex -shell-escape summary.tex` (requires pygments for minted).